

Semana 13

Passo 1 – Criar a Tabela CLIENTES

Primeiro criamos a tabela que armazenará os dados dos clientes.

```
CREATE TABLE clientes (  
    id_cliente INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL  
);
```

Explicação

- **CREATE TABLE** → cria uma nova tabela.
 - **id_cliente** → identificador único do cliente.
 - **PRIMARY KEY** → chave primária.
 - **nome** → nome do cliente.
 - **NOT NULL** → campo obrigatório.
-

Passo 2 – Criar a Tabela PEDIDOS

Agora criamos a tabela dos pedidos.

```
CREATE TABLE pedidos (  
    id_pedido INT PRIMARY KEY,  
    id_cliente INT,  
    valor_total DECIMAL(10,2),  
    status VARCHAR(20),  
    FOREIGN KEY (id_cliente)  
    REFERENCES clientes(id_cliente)  
);
```

Explicação

- `id_pedido` → código único do pedido.
 - `id_cliente` → cliente que realizou o pedido.
 - `valor_total` → valor do pedido.
 - `status` → situação do pedido.
 - `FOREIGN KEY` → cria o relacionamento com a tabela clientes.
-

Passo 3 – Verificar se as Tabelas Foram Criadas

No MySQL:

```
SHOW TABLES;
```

Para visualizar a estrutura:

```
DESCRIBE clientes;
```

```
DESCRIBE pedidos;
```

Passo 4 – Inserir Clientes

```
INSERT INTO clientes (id_cliente, nome)
```

```
VALUES (1, 'Ana');
```

```
INSERT INTO clientes (id_cliente, nome)
```

```
VALUES (2, 'Carlos');
```

Verificar:

```
SELECT * FROM clientes;
```

Resultado:

id_cliente	nome
------------	------

1	Ana
---	-----

2	Carlos
---	--------

Passo 5 – Inserir Pedidos

INSERT INTO pedidos

(id_pedido, id_cliente, valor_total, status)

VALUES

(101, 1, 250.00, 'Aberto');

INSERT INTO pedidos

(id_pedido, id_cliente, valor_total, status)

VALUES

(102, 2, 500.00, 'Aberto');

Verificar:

SELECT * FROM pedidos;

Resultado:

id_pedido	id_cliente	valor_total	status
-----------	------------	-------------	--------

101	1	250.00	Aberto
-----	---	--------	--------

102	2	500.00	Aberto
-----	---	--------	--------

Passo 6 – Atualizar um Pedido

Alterar o status do pedido 101:

```
UPDATE pedidos
```

```
SET status = 'Concluído'
```

```
WHERE id_pedido = 101;
```

Consultar:

```
SELECT * FROM pedidos;
```

Passo 7 – Excluir um Pedido

Excluir o pedido 102:

```
DELETE FROM pedidos
```

```
WHERE id_pedido = 102;
```

Consultar:

```
SELECT * FROM pedidos;
```

Passo 8 – Criar uma Tabela de Auditoria

A auditoria armazenará registros automáticos feitos pela trigger.

```
CREATE TABLE auditoria (
```

```
    id_auditoria INT AUTO_INCREMENT PRIMARY KEY,
```

```
    id_pedido INT,
```

```
data_alteracao DATETIME,  
mensagem VARCHAR(100)  
);
```

Passo 9 – Criar a Trigger

Sempre que um pedido for atualizado, a trigger registrará a alteração.

MySQL

```
DELIMITER //
```

```
CREATE TRIGGER atualiza_pedido  
AFTER UPDATE ON pedidos  
FOR EACH ROW  
BEGIN  
    INSERT INTO auditoria  
    (id_pedido, data_alteracao, mensagem)  
    VALUES  
    (NEW.id_pedido, NOW(), 'Pedido atualizado');  
END//
```

```
DELIMITER ;
```

Passo 10 – Testar a Trigger

Atualize um pedido:

```
UPDATE pedidos  
SET status = 'Entregue'
```

WHERE id_pedido = 101;

Consultar auditoria:

SELECT * FROM auditoria;

Resultado esperado:

id_auditoria	id_pedido	data_alteracao	mensagem
1	101	2026-06-01	Pedido atualizado

Resumo dos Comandos Utilizados

Comando	Função
CREATE TABLE	Criar tabela
INSERT INTO	Inserir dados
SELECT	Consultar dados
UPDATE	Atualizar dados
DELETE	Excluir dados
FOREIGN KEY	Criar relacionamento
CREATE TRIGGER	Criar automação

SHOW TABLES

Listar tabelas

DESCRIBE

Ver estrutura da
tabela